

Snooker Fantasy League: A Reverse-Engineering Course

Snooker Fantasy League: A Reverse-Engineering Course

A nine-lesson walk-back from a finished 1,654-line React app to the first commit it would have come from. Read alongside `_source/SnookerFantasyLeague.jsx` open in another window.

Confirming the source

I read `SnookerFantasyLeague.jsx` end-to-end. The five top-level **data constants** are:

1. `ROUND1_MATCHES` — 16 first-round matches
2. `ROUND2_MATCHES` — 8 last-16 matches
3. `QF_MATCHES` — 4 quarter-finals
4. `SF_MATCHES` — 2 semi-finals
5. `FINAL_MATCH` — 1 final

...with two more big data structures that the app leans on: `TEAMS` (the eight fantasy teams and their picks) and `PLAYER_INFO` (per-player metadata, keyed by name).

The five **tabs** are:

1. **Standings** — leaderboard + headline stats
2. **Matches** — match cards per round, finished/not-finished
3. **Predictions** — comparison matrix + single-team card
4. **Players** — 32-player grid + a deep player-detail view
5. **Analytics** — Recharts dashboards (points, accuracy, agreement, insights)

All set. Course starts below. Each lesson ends with a **Vibe prompt you would have used** section and a **CHECK YOURSELF** box.

Lesson 0 — The Problem (no code)

Imagine the pub conversation

You and seven mates watch the World Snooker Championship every May. Sheffield, Crucible, two weeks of green-baize warfare. Someone — probably you — says: “*We should make a fantasy league out of this.*” Everyone agrees. Pints are raised. Now what?

A snooker tournament has a fixed **bracket of 32 players**. They play through five rounds: Round 1 (last 32), Round 2 (last 16), Quarter-Finals (last 8), Semi-Finals (last 4), and the Final. The bracket is published before a ball is potted. So **every match is known in advance** — only the winners are unknown.

That’s the whole hook. If every match is known, then every team can submit picks for every match in every round, **before the tournament starts**. As results come in, the league scores those picks.

The 8 teams

There are eight fantasy teams: *Invincibles*, *Uncredibles*, *Hopeless*, *Clueless*, *Break Builders United*, *The Untouchables*, *Selbies*, *One Four Sevens*. Each one is a person (or a couple) sitting at home with their bracket, picking who they think wins each match. Each team needs:

- 16 picks for Round 1 (one per match)
- 8 picks for Round 2
- 4 picks for the Quarter-Finals
- 2 picks for the Semi-Finals
- 1 pick for the Final

That’s **31 picks per team, times 8 teams = 248 picks total**, all locked in before the first break-off.

The scoring rules (3 bullets)

- **3 points** if your pick wins the match.
- **1 point** (a “consolation”) if your pick loses — you still showed up.

- **null / pending** if the match hasn't been played yet — neither right nor wrong, just not scored.

That last point matters more than it looks. Lesson 2 is built around it.

What the app needs to do

When the tournament is live, you want to be able to: see who's leading, see how each round shook out, drill into one team's picks, drill into one player's appearances, and see charts of who agreed and who didn't. That's it. **No login, no database, no sharing.** It's a static page; the data is edited by hand as results come in.

That gives the architecture away already: it's a single-page React app whose entire "database" is a few JavaScript constants. Lesson 1 starts there.

Vibe prompt you would have used

"I want to build a single-page React app for a fantasy snooker league. Eight teams pick winners across five rounds (R1: 16 picks, R2: 8, QF: 4, SF: 2, Final: 1). Scoring: 3 points if the pick wins, 1 if it loses, null while the match is unplayed. No database — the bracket and team picks are edited by hand in code. The page should have tabs for Standings, Matches, Predictions, Players, and Analytics. Don't write any code yet — just confirm you understand the problem and ask me anything that's ambiguous."

That last sentence ("don't write any code yet — just confirm") is the single most useful trick in this lesson. It forces the LLM to surface ambiguities you didn't think of.

CHECK YOURSELF

1. Why does scoring use **null** for unplayed matches instead of **0**?
 2. Why are there exactly **31 picks per team**, no more and no less?
 3. If a 9th friend wanted to join, what's the minimum data change required? (Hint: it's a one-line addition.)
-

Lesson 1 — The Shape of the Data (data modeling, no UI yet)

Before any pixel renders, you need to decide what shapes you're pushing around. In a Python data-engineering background this is the part that feels comfortable — it's a schema design problem, just expressed in JS objects instead of pydantic models.

What's a Match?

Every match in the tournament is the same shape:

```
{ id: 1, p1: 'Zhao Xintong', p2: 'Liam Highfield', winner: 'Zhao Xintong', score: '10-7', seed1: 1 }
```

Five fields, three of which are obvious (id, p1, p2) and two of which are the interesting ones:

- winner is **either a string or undefined**. Undefined means *the match hasn't been played*. As soon as a result comes in, you set winner to one of the two player names. This is the field that drives the scoring logic.
- score is a display-only string like '10-7' or '17-18'. The app never parses it for points; it only splits it on '-' to render two big numbers next to the players.
- seed1 is on R1 only, because that's the only round where the bracket is constructed by seed; later rounds the seed is implied by the path.

The app declares the matches up-front in five flat arrays — one per round — in match order:

```
const ROUND1_MATCHES = [
  {
    id: 1,
    p1: 'Zhao Xintong',
    p2: 'Liam Highfield',
    winner: 'Zhao Xintong',
    score: '10-7',
    seed1: 1,
  },
  { id: 2, p1: 'Judd Trump', p2: 'Gary Wilson', winner: 'Judd Trump', score: '10-7', seed1: 2 },
  // ... 14 more
];
```

The match order is **the bracket order**, top to bottom. R1 has 16 entries, R2 has 8, QF has 4, SF has 2, Final has 1 — the exact halving you'd expect from a knockout draw.

What's a Team?

A team is a name + a vibe + five arrays of picks:

```
{
  name: 'Invincibles',
  color: '#0F5132', accent: '#10B981',
  icon: '🏆', motto: 'Unstoppable & Unbeatable',
  r1: ['Zhao Xintong', 'Judd Trump', /* 14 more */], // 16 picks
  r2: ['Zhao Xintong', 'Xiao Guodong', /* 6 more */], // 8 picks
  qf: ['Zhao Xintong', 'Mark Allen', 'John Higgins', 'Wu Yize'], // 4 picks
  sf: ['Shaun Murphy', 'Wu Yize'], // 2 picks
  final: 'Wu Yize', // 1 pick (no array)
}
```

The color / accent / icon / motto are pure cosmetics — they drive every gradient header and team chip in the UI. The five pick arrays are where the actual game lives.

final is a single string instead of a one-element array. That asymmetry is on purpose: there's only one Final, and it's nicer to write `team.final === 'Wu Yize'` than `team.final[0] === 'Wu Yize'`. The cost is that the scoring code has one branch for final and another for everything else (you'll see this in `calculateTeamScores`).

What's PLAYER_INFO and why is it an object keyed by name?

```
const PLAYER_INFO = {
  'Zhao Xintong': { country: '🇨🇳', seed: 1, status: 'Defending Champion', flag: 'CN' },
  'Judd Trump': { country: '🇬🇧', seed: 2, status: 'World No. 1', flag: 'ENG' },
  // ... 30 more
};
```

It's an object (a dictionary, a map) keyed by **player name** — not an array. Why? Because every other place in the app already has a player's name as a string (from a match's p1/p2, or from a team's pick), and the most common operation is “given a name, get the metadata for that player.” That's a hash lookup: `PLAYER_INFO['Zhao Xintong']`. If it were an array of { name, country, seed, ... } objects, you'd be writing `.find(p => p.name === name)` everywhere — $O(n)$ instead of $O(1)$, and uglier to read.

The Python equivalent is the same instinct that makes you reach for `dict[str, Play-`

erInfo] instead of list[PlayerInfo].

The most important rule in the codebase

Here is the invariant that the entire scoring engine depends on:

For each round, the i -th element of team.r1 (or r2, or qf, or sf) corresponds to the i -th element of ROUND1_MATCHES (or ROUND2_MATCHES, ...).

Index 0 of team.r1 is the team's pick for ROUND1_MATCHES[0]. Index 1 is the pick for match 1. And so on. There is no matchId field on the picks. There is no key. **Order is the contract.**

That's why scorePick can be the trivial five-line function it is in Lesson 2 — it just takes a pick and a match and asks “did the pick equal the match's winner?” If you ever re-order ROUND1_MATCHES without re-ordering all eight teams' r1 arrays in lockstep, every score in Round 1 silently goes wrong. There is no warning, no test, no type system catching this — it's a convention enforced by the file.

If you grew up writing SQL, this feels barbaric (“where's the foreign key?”). It's barbaric. It's also fine for an 8-team league where one person owns the file. The moment a 9th team or 33rd player shows up, this is the first thing that needs schema-ing.

Vibe prompt you would have used

“Here's the data model for my fantasy snooker league. Don't generate any UI yet. I want to define the constants. Match: { id, p1, p2, winner?, score?, seed1? }. Team: { name, color, accent, icon, motto, r1[16], r2[8], qf[4], sf[2], final } where each pick array is in the same order as the matching round's match array. PlayerInfo: an object keyed by player name with { country, seed, status, flag }. Generate empty constants for ROUND1_MATCHES through FINAL_MATCH (5 arrays), an empty TEAMS array of 8 with placeholder picks, and a PLAYER_INFO object stub with the 16 seeded players. Add a comment above the team picks reminding the reader: 'pick array index i corresponds to match index i of the matching round.'”

CHECK YOURSELF

1. If you swapped match 7 and match 8 in `ROUND1_MATCHES` but forgot to swap them in every team's `r1` array, what would visibly go wrong in the UI?
 2. Why is `PLAYER_INFO` keyed by name and not by seed?
 3. `team.final` is a string but `team.sf` is an array. What's the smallest code change that would make them consistent, and what would it cost in readability?
-

Lesson 2 — The Pure Logic (scoring, before any React)

This is the smallest, most important file in the project. It's about 40 lines and it's the **heart** of the app — every UI screen is just a different lens on its output.

Why scoring lives outside React

The original file declares two functions before the first JSX tag:

```
function scorePick(pick, match) {  
  if (!match || !match.winner) return null;  
  if (pick === match.winner) return 3;  
  return 1;  
}  
  
function calculateTeamScores(team) {  
  /* ... */  
}
```

Neither of them imports React. Neither knows what a component is. They take plain data in and return plain data out. That's deliberate. **Pure functions are the part of your codebase that survives every refactor.** When Phase 1 of this project moves the app to Next.js, these two functions are copied into `lib/scoring.ts` essentially unchanged — the components around them are rewritten, but `scorePick` is the same.

The Python instinct here is right: this is the “business logic” layer. Keep it free of framework concerns. UIs come and go; the rule that picking-the-winner = 3 points doesn't.

Walking through scorePick line by line

```
function scorePick(pick, match) {  
  if (!match || !match.winner) return null; // pending  
  if (pick === match.winner) return 3; // correct  
  return 1; // wrong (consolation)  
}
```

Three branches:

1. `!match || !match.winner` → `null`. If the match doesn't exist (defensive) or hasn't been played yet (winner is undefined), the pick can't be scored. The function returns `null`, **not 0** and **not false**. That distinction is the heart of the round.
2. `pick === match.winner` → `3`. Right pick.
3. **Else** → `1`. Wrong pick, but you participated, so 1 point.

Why correct is null, not false

Inside `calculateTeamScores` you'll see this pattern repeated for every round:

```
const r1Details = team.r1.map((pick, i) => {  
  const pts = scorePick(pick, ROUND1_MATCHES[i]);  
  r1 += pts || 0;  
  return { match: ROUND1_MATCHES[i], pick, points: pts, correct: pts === null  
});
```

Look at the last expression: `correct: pts === null ? null : pts === 3`.

`correct` has **three states**, not two:

- `true` — the match is finished and the pick was right.
- `false` — the match is finished and the pick was wrong.
- `null` — the match isn't finished yet.

The UI uses this directly. In `<PicksList>`:

```
const pickPending = d.points === null;  
// ...  
background: pickPending ? '#F9FAFB' : d.correct ? '#DCFCE7' : '#FEE2E2',
```

Three colors: gray for pending, green for correct, red for wrong. If correct were just a

boolean, “pending” and “wrong” would render as the same red badge — and during the tournament, when most matches are still upcoming, the screen would look catastrophic.

This is the first lesson the brief tells you to internalize: **tri-state booleans aren’t a code smell, they’re a feature**. The other place this comes up is `match.winner` itself — undefined means “not played”, a string means “played and winner is this person”. Those are not the same as `null` would be in some languages; the absence of a winner is a real first-class state.

`r1 += pts || 0` — the running total trick

That line is doing two things at once:

```
r1 += pts || 0;
```

If `pts` is 3 or 1, you add it. If `pts` is `null` (pending), `null || 0` evaluates to 0 and you add nothing. So `r1` accumulates only finished, scored points. The total never goes “wrong” while a round is in progress; it just lags reality until results land.

What `calculateTeamScores` returns

The return shape is wide on purpose:

```
return {  
  r1,  
  r2,  
  qf,  
  sf,  
  f,  
  total, // 6 numbers  
  r1Details,  
  r2Details,  
  qfDetails,  
  sfDetails,  
  fDetails, // 5 arrays of {match, pick, points, correct}  
};
```

Six numbers (the per-round totals + the grand total) plus five arrays of the per-pick detail objects. The numbers drive the leaderboard column totals. The detail arrays drive the

predictions matrix, the team card view, the player detail page — anywhere the app says “here’s pick X against match Y, was it right, how many points.” **Compute once, render five different ways.**

Inside the orchestrator component you’ll see exactly that:

```
const teamsWithScores = useMemo(() => {
  return TEAMS.map((t) => ({ ...t, scores: calculateTeamScores(t) })).sort(
    (a, b) => b.scores.total - a.scores.total
  );
}, []);
```

Every team is decorated with its scores object once, and that decorated array is then handed down to every tab. The five tabs share the same computation; none of them re-invoke `calculateTeamScores` themselves.

Vibe prompt you would have used

*“I have my data constants set up (ROUND1_MATCHES through FINAL_MATCH, plus a TEAMS array with r1[16]/r2[8]/qf[4]/sf[2]/final picks). Write me two pure functions, no React, no JSX. `scorePick(pick, match)` returns 3 if `pick === match.winner`, returns 1 if the match has a winner but the pick was wrong, and returns null if `match.winner` is undefined. Then `calculateTeamScores(team)` walks through each round’s pick array and the matching match array in parallel by index, sums the points (treating null as 0 for the running total), and returns { r1, r2, qf, sf, f, total, r1Details, r2Details, qfDetails, sfDetails, fDetails } where each *Details is an array of { match, pick, points, correct } and correct is null if pending, otherwise points === 3.”*

CHECK YOURSELF

1. If `scorePick` returned 0 instead of null for pending matches, what’s the first thing that would visibly break in the UI? (Hint: `.filter(d => d.correct).length` is used a lot.)
2. Why is `calculateTeamScores` called inside `useMemo` instead of inline in the JSX?
3. Which property of the return value of `calculateTeamScores` do the **standings columns** read, and which property do the **predictions matrix cells** read? Why are

both needed?

Lesson 3 — The First Render (a single component, no tabs yet)

This is the part that vibe coders get wrong on day one: trying to build the whole app in the first prompt. Don't. **A vibe-coded app is grown one feature at a time, starting with the smallest thing that proves the data is real.** For Snooker Fantasy League, that thing is a list of team names with their total points.

v1: just team names and totals

If you go back to the very beginning of the project, the first version of `<StandingsTab>` would have been about 15 lines:

```
function StandingsTab({ teams }) {
  return (
    <table>
      <thead>
        <tr>
          <th>Rank</th>
          <th>Team</th>
          <th>Total</th>
        </tr>
      </thead>
      <tbody>
        {teams.map((team, i) => (
          <tr key={team.name}>
            <td>{i + 1}</td>
            <td>{team.name}</td>
            <td>{team.scores.total}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

```
}
```

That's it. No styles, no icons, no gradients. The point is to **prove that the data flow works**: `calculateTeamScores` runs, the totals compute, `.sort()` orders the rows, and React renders eight rows. If any one of those is wrong, you find out in the first 30 seconds, not in hour three after you've added a hero header and tab bar.

Once that's on screen you have your foothold. Everything afterwards is "now add a column."

v2: per-round columns

The next thing a human builder would notice is *"I can't tell which team is doing well in which round."* So you add R1 and R2 columns:

```
<th>Rank</th><th>Team</th>
<th>R1</th>
<th>R2</th>
<th>Total</th>
```

```
<td>{team.scores.r1}</td>
<td>{team.scores.r2}</td>
```

This is a one-line-prompt change ("add a column for R1 score next to the team name"). You don't redesign the table; you just add cells. The fact that `team.scores` already has `r1` and `r2` waiting is exactly why Lesson 2's "wide return shape" was worth it.

v3: progressive enhancement of the same column

Then the tournament progresses and you add `qf`, `sf`, and `final`. By the end of the tournament, the row in the production code looks like the one in the file:

```
<td style={{ ...td, textAlign: 'center' }}>
  <div style={{ fontWeight: 700, fontSize: 17, color: '#0F5132' }}>{team.score}
  <div style={{ fontSize: 11, color: '#6B7280' }}>/ 48</div>
</td>
```

Two divs, not just a number — the team's R1 score over the maximum possible 48 (16 matches × 3 points). That `/48` line was almost certainly added in a separate prompt:

“under the R1 number, show ‘/ 48’ in a smaller gray font so I can see how close they are to a perfect round.” This is the right grain for a vibe prompt — small, specific, low-risk.

v4: the special “Final Pick” cell

The Final column is different from the others. It’s not a number — it’s a **chip** showing who the team picked, color-coded by whether the actual champion matched:

```
<td style={{ ...td, textAlign: 'center' }}>
  <div
    style={{
      display: 'inline-block',
      padding: '4px 10px',
      borderRadius: 12,
      background: team.final === 'Wu Yize' ? '#FEE2E2' : '#DBEAFE',
      color: team.final === 'Wu Yize' ? '#991B1B' : '#1E40AF',
      fontWeight: 700,
      fontSize: 12,
    }}
  >
    {team.final}
  </div>
</td>
```

Notice that the colors here are **hard-coded** to “if the pick is Wu Yize, red — otherwise blue.” That’s a tell that this code was written **after the final result was known**. During the tournament, when the champion wasn’t yet decided, this branch would have looked completely different — probably just neutral coloring. The hard-coded ‘Wu Yize’ is a tournament-end snapshot. A more general version would compare against `FINAL_MATCH[0]?.winner`. The code is one prompt away from being made tournament-agnostic.

This is a normal pattern in vibe-coded codebases: **branches that were once dynamic become hard-coded after the data is final**. Not technically wrong, but worth knowing it’s there.

v5: the leader highlight

The leader row deserves a crown. So somewhere mid-project a prompt added:

```
const rankColor = rank === 1 ? '#FBBF24' : rank === 2 ? '#9CA3AF' : rank === 3 ? '#D9EAD3' : '#A6A6A6';
// ...
<div style={{ ... background: rankColor, ... }}>
  {rank === 1 ? '👑' : rank}
</div>
```

Gold for 1st, silver for 2nd, bronze for 3rd, gray otherwise. The leader gets a crown emoji instead of “1”. That’s two visual rules added in one prompt; you can see how a single sentence (“medal-color the rank circle and replace 1st place’s number with a crown emoji”) becomes those four lines.

v6: the form badge

Last, the rightmost column tracks form — whether the team is doing better in R2 than they did in R1. That’s `<FormBadge>`:

```
function FormBadge({ r1Pct, r2Pct }) {
  const trending = r2Pct > r1Pct ? 'up' : r2Pct < r1Pct ? 'down' : 'flat';
  const Icon = trending === 'up' ? TrendingUp : trending === 'down' ? TrendingDown : TrendingFlat;
  // ... renders an arrow + label
}
```

It’s a tiny presentational component. Notice it’s a **separate function**, not inlined into the table row. That’s because you’ll want it again in player cards, in summary tiles, anywhere you want to express “this is going up / down / sideways.” Lesson 5 will come back to this idea: when a thing has a name, give it a function.

Vibe prompt you would have used (at each step)

v1: “Render the eight teams as a table sorted by `team.scores.total`. Three columns: Rank, Team, Total. Plain HTML, no styles yet, just verify the totals are right.”

v2: “Add R1 and R2 columns next to Team showing `team.scores.r1` and `team.scores.r2`.”

v3: “Under each round score in the table, show the maximum possible like ‘/ 48’ for R1 and ‘/ 24’ for R2 in a smaller gray font.”

v4: “Add a *Final Pick* column. Render *team.final* inside a rounded chip with a 🏆 emoji. If the pick equals the eventual champion (Wu Yize), use a red background; otherwise blue.”

v5: “Make the rank circle gold for 1st, silver for 2nd, bronze for 3rd. Show a 🥇 emoji instead of the number for 1st place.”

v6: “Add a *Form* column at the right that compares the team’s R1 percentage to their R2 percentage. If they’re improving show a green up-arrow and ‘Rising’; if dropping, red down-arrow ‘Falling’; if flat, gray circle ‘Steady’. Extract it to a `<FormBadge>` component.”

The shape is always the same: **one column, one widget, one prompt**. Never “redesign the standings table” — that’s the kind of prompt that produces 800 lines of speculative CSS and breaks something invisible.

CHECK YOURSELF

1. The Final Pick chip currently hard-codes 'Wu Yize' to choose its color. What’s the one-line edit to make it tournament-agnostic?
2. Why is `<FormBadge>` extracted into its own function instead of inlined into the table cell?
3. If you wanted to add a new “% correct” column between R2 and Total, what minimum changes would the prompt need to specify? (Hint: it’s two cells per row plus one header.)

Lesson 4 — State & Tabs (introducing `useState`)

Up until now we’ve talked about the app like it’s one screen. It isn’t — it’s five tabs that share data. The moment you have two screens that show the *same* data with *different* lenses, you need a top-level component that owns the state and feeds it down. This is the React mental model that breaks the most for newcomers, so it’s worth getting right.

Two pieces of state, one home

The orchestrator component is `<SnookerFantasyLeague>`. Its top three lines:

```
export default function SnookerFantasyLeague() {
  const [activeTab, setActiveTab] = useState('standings');
  const [selectedTeam, setSelectedTeam] = useState(null);
}
```

Two pieces of state. **activeTab** is a string like 'standings' or 'predictions' — which of the five tabs is currently visible. **selectedTeam** is either null (nobody clicked yet) or one of the team objects — used by the Predictions tab to know which team's deep card to show.

These two pieces of state cannot live inside the individual tab components, because **clicking a row in <StandingsTab> needs to switch the visible tab to <PredictionsTab> and remember which team to highlight**. That cross-tab effect needs a place where both tabs can read and write the same state. That place is the parent.

The pattern is:

State up, callbacks down. The state lives at the highest component that needs to see all of it. The leaf components don't own state; they receive props and call back when they want a change.

useMemo for the decorated team list

The next line is the unsung hero of the whole file:

```
const teamsWithScores = useMemo(() => {
  return TEAMS.map((t) => ({ ...t, scores: calculateTeamScores(t) })).sort(
    (a, b) => b.scores.total - a.scores.total
  );
}, []);
```

Read it inside-out:

- `TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t) }))` — for each of the 8 teams, spread its fields (name, color, r1[], ...) and add a fresh scores object computed from `calculateTeamScores`.
- `.sort((a, b) => b.scores.total - a.scores.total)` — descending by total points so index 0 is the leader.
- Wrapped in `useMemo(..., [])` — compute this **once** on first render and reuse forever.

Why useMemo? Because **without it, every time the user clicks a tab, React would recompute all 8 teams' scores from scratch**. That's $8 \times 31 = 248$ scorePick calls per click. It's not slow enough to feel, but it's wasteful, and more importantly it would mean a new teamsWithScores array reference on every render — which can cascade into spurious re-renders of every tab that receives it as a prop.

The empty dependency array [] says: *"the inputs don't change during the lifetime of the component — TEAMS is a module-level constant — so cache the result once."* If TEAMS were ever loaded from an API and stored in state, that array would have to include the state variable.

This is the same reflex as memoizing an expensive .transform() step in a Pandas pipeline. The data layer's free; the UI layer should look it up, not recompute it.

Wiring the tabs

After the state and the memoized data, the component renders three things:

1. The hero header (decorative — no state).
2. A horizontal nav bar that maps over a tabs array and renders five buttons. Clicking any button calls setActiveTab(tab.id).
3. The content area, which **switches on activeTab** and renders the right tab component:

```
{
  activeTab === 'standings' && (
    <StandingsTab
      teams={teamsWithScores}
      onTeamClick={(t) => {
        setSelectedTeam(t);
        setActiveTab('predictions');
      }}
    />
  );
}
{
  activeTab === 'matches' && <MatchesTab />;
}
```

```

{
  activeTab === 'predictions' && (
    <PredictionsTab
      teams={teamsWithScores}
      selectedTeam={selectedTeam}
      setSelectedTeam={setSelectedTeam}
    />
  );
}
{
  activeTab === 'players' && <PlayersTab teams={teamsWithScores} />;
}
{
  activeTab === 'analytics' && <AnalyticsTab teams={teamsWithScores} />;
}

```

Three things to notice:

- **<MatchesTab /> takes no props.** It only uses the static match constants from `lib/matches.ts`. There's no team data flowing in.
- **<StandingsTab> receives a callback `onTeamClick`** that does *two things at once*: it sets the selected team **and** switches the tab. That's the cross-tab interaction the parent had to coordinate. A child component couldn't have done this on its own.
- **<PredictionsTab> receives both the read (`selectedTeam`) and the write (`setSelectedTeam`).** This is the pattern that gets called "controlled component": the parent owns the source of truth; the child renders against it and asks the parent to update it.

Why not put `selectedTeam` inside `<PredictionsTab>`?

Because then the `<StandingsTab>`'s row click couldn't reach it. The state has to live at the **lowest common ancestor** of every component that needs to read or write it. For `selectedTeam`, the readers are `<PredictionsTab>` and the writers are both `<StandingsTab>` (when you click a row) and `<PredictionsTab>` (when you click a team chip inside it). Their lowest common ancestor is `<SnookerFantasyLeague>` itself, so that's where the state lives.

Vibe prompt you would have used

"Wrap my single <StandingsTab> in a top-level <SnookerFantasyLeague> component. Add a useState for activeTab (default 'standings') and a sticky horizontal nav bar with five tabs: Standings, Matches, Predictions, Players, Analytics, each with a Lucide icon. Render the matching tab component below. Add a second useState for selectedTeam (default null). Pass teamsWithScores (a useMemo'd sort of TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t) }))) by descending total) to every tab that needs it. From <StandingsTab>, when a row is clicked, set selectedTeam and switch activeTab to 'predictions'."

CHECK YOURSELF

1. If you removed useMemo and just wrote `const teamsWithScores = TEAMS.map(...)` directly, what specifically would re-run on every tab switch?
 2. Why does setSelectedTeam get passed down to <PredictionsTab> as a prop instead of being created with its own useState inside that tab?
 3. Why does <MatchesTab /> not need any props at all, while <StandingsTab> needs two?
-

Lesson 5 — Composition & Prop Drilling (when to split a component)

A 200-line component is a smell. So is splitting too early. The right time to extract a sub-component is when **either** of these is true:

- You're about to copy-paste a chunk of JSX and tweak one variable.
- A section is conceptually about a different "thing" than the rest of the component.

Both of those happen all over this codebase. The clearest example is the chain <PlayersTab> → <PlayerDetail> → <MatchPickAnalysis> → <TeamChip>.

The chain, explained

<PlayersTab> (top of the players folder) has two jobs: render the 32-player grid, and — when a player is clicked — render <PlayerDetail> for that player. The rule "two jobs" is

fine in a parent component; that's literally what an orchestrator does. The actual rendering of those two jobs is delegated:

```
if (selectedPlayer) {  
  return <PlayerDetail playerName={selectedPlayer} teams={teams} onBack={...}  
}  
// ... otherwise render the grid of <PlayerCard> buttons
```

<PlayerDetail> (about 270 lines of computation + JSX) does **all** the per-player math: which matches did this player appear in, who picked them, who picked against them, did they win, what's their tournament path. That's a lot. So inside it, the per-match analysis is delegated:

```
{  
  pickAnalyses.map((pa, i) => (  
    <MatchPickAnalysis  
      key={i}  
      analysis={pa}  
      playerName={playerName}  
      onSelectPlayer={onSelectPlayer}  
    />  
  ));  
}
```

<MatchPickAnalysis> renders one match (e.g. "Murphy vs Wu Yize, Final, Wu won 18-17") and the two columns of teams that backed each side. Inside it, the team chips are delegated:

```
{  
  teamsBackedThis.map((t) => <TeamChip key={t.name} team={t} ptsEarned={ptsFor}  
}
```

<TeamChip> is **15 lines**. It's a single colored pill showing the team icon, name, and the points its picker earned. Why does it deserve its own file? Because it's drawn **twice on every single match analysis** — once in the "Backed Murphy" column and once in the "Backed Wu" column — and copy-pasting 15 lines of JSX twice would be an obvious smell. Extract.

When to extract: the copy-paste rule

If you find yourself writing the same JSX twice with minor tweaks, that's the signal. Inside `<MatchPickAnalysis>`:

```
<div>
  <div>BACKED {playerName.toUpperCase()}</div>
  {teamsBackedThis.map(t => <TeamChip team={t} ptsEarned={ptsForPicker} />)}
</div>
<div>
  <div>BACKED {opponent.toUpperCase()}</div>
  {teamsBackedOpponent.map(t => <TeamChip team={t} ptsEarned={ptsForOpponent} />)}
</div>
```

Two columns, identical structure, different inputs. The chip is extracted; the **column wrapper is not** — because it's only used twice and only in this file, and the difference (color theme, points value) is enough that a third level of abstraction would be premature. **The right amount of abstraction is the smallest amount that keeps you from copy-pasting.**

Tiny components that exist for readability, not reuse

`<PathStep>` and `<PathArrow>` are a different breed.

```
<PathStep label="R1" status={r1Won ? 'won' : 'lost'} />
<PathArrow />
<PathStep label="R2" status={r2Won ? 'won' : 'lost'} />
<PathArrow />
<PathStep label="QF" status={qfFinished ? (qfWon ? 'won' : 'lost') : 'live'} />
<PathArrow />
<PathStep label="SF" status={...} />
<PathArrow />
<PathStep label="F" status={...} />
```

`<PathArrow>` is literally one character: `→`. Why is it a component? **Because the JSX above reads like a sentence.** If you inlined the arrow's styling, you'd see five identical `<span style={{ color: ..., fontSize: 18 }}>→</span>` blocks interrupting the flow. With the component, the markup tells you the structure at a glance: step, arrow, step, arrow, step. That's the "tournament path" widget, and the JSX makes that obvious. Extracting `<PathArrow>` cost nothing and bought one of the easier-to-read snippets in

the file.

`<PathStep>` is the same idea at slightly larger scale: the four visual states (won / lost / live / na) are mapped from a status string to a small style object inside the component. That logic could have been inlined, but then the parent would carry five copies of a `if (status === 'won') ... else if (status === 'lost') ...` block. By moving it inside `<PathStep>`, the parent declares **what it wants** (“step labelled R1, status ‘won’”) instead of **how to render it**. That’s the canonical reason to make a component: separating *what* from *how*.

Prop drilling: when it’s fine

teams gets passed from `<SnookerFantasyLeague>` to `<PlayersTab>` to `<PlayerDetail>` to `<MatchPickAnalysis>`. Three levels. That’s “prop drilling” and at some point newcomers ask “shouldn’t this be a Context or a Redux store?”

For an app with one shared array and three levels of nesting? **No**. Context exists to spare you from passing the **same** prop through five-plus levels of unrelated components. Three levels of clearly named props is **cheaper to reason about** than introducing a global. The day you have ten teams’ worth of state plus user auth plus filters plus a settings panel, you re-evaluate. Until then, `<Component teams={teams}>` is fine and the file will tell you.

Vibe prompt you would have used

“This `<MatchPickAnalysis>` component is getting long. Extract two pieces. First, the colored pill that shows a team’s icon, name, and points-earned chip – call it `<TeamChip>`, props { team, ptsEarned }. Second, the small status step in the tournament-path bar – call it `<PathStep>`, props { label, status } where status is 'won' | 'lost' | 'live' | 'na', each mapping to its own colors and icon (🟩 🟦 ● →). Also extract the literal → arrow into a one-line `<PathArrow>` component so the path JSX reads like ‘step arrow step arrow step’. Don’t change any visuals – just move the JSX into the new files and import them back.”

That last sentence (“don’t change any visuals, just move the JSX”) is the prompt that turns a refactor from a rewrite back into a refactor. Vibe coders have to add it explicitly; LLMs love to redesign while they’re moving things.

CHECK YOURSELF

1. `<PathArrow>` has zero props and renders one character. What's the argument *for* keeping it as its own component, even though it could trivially be inlined?
 2. Why is the **column wrapper** inside `<MatchPickAnalysis>` not extracted, even though it appears twice with similar structure?
 3. At what nesting depth would you reach for Context instead of prop drilling? Why isn't this app there yet?
-

Lesson 6 — Charts & Data Transforms (Recharts)

The trick most React-charting tutorials skip is the part that actually matters: **charts in React are about data shape, not chart libraries**. Recharts (or Victory, or any other) takes an array of plain objects with a known schema and draws bars/lines/cells from it. Your job, 90% of the time, is to *reshape your existing data into the schema the chart wants*. The other 10% is choosing axes and colors.

Look at the Analytics tab. Almost every chart in it begins with a `teams.map(...)` that produces a brand new array. That array is the chart's "data."

Reshape #1: stacked-bar totals

```
const r1Data = teams.map((t) => ({
  team:
    t.name.length > 12
      ? t.name
        .split(' ')
        .map((w) => w[0])
        .join('')
      : t.name,
  fullName: t.name,
  R1: t.scores.r1,
  R2: t.scores.r2,
  QF: t.scores.qf,
  SF: t.scores.sf,
```

```

    Final: t.scores.f,
    Total: t.scores.total,
  }));

```

Each row is one team. The team field is the X-axis label (abbreviated to initials if the name is too long). Then four numeric fields — R1, R2, QF, SF, Final — that become the **stacks**. Recharts' `<Bar dataKey="R1" stackId="a" />` reads the R1 field of every row; the `stackId="a"` tells it to stack on top of the previous bar with the same `stackId`. Same `stackId` across multiple `<Bar>` elements = a single stacked bar per row:

```

<Bar dataKey="R1"      stackId="a" fill="#0F5132" name="Round 1" />
<Bar dataKey="R2"      stackId="a" fill="#DC2626" name="Round 2" />
<Bar dataKey="QF"      stackId="a" fill="#FBBF24" name="Quarter-Finals" />
<Bar dataKey="SF"      stackId="a" fill="#7C3AED" name="Semi-Finals" />
<Bar dataKey="Final"   stackId="a" fill="#9F1239" name="Final" radius={[4, 4, 0, 0]} />

```

If you change `stackId` to a different value (or remove it), the same data renders as **grouped** bars (R1, R2, QF side-by-side per team) instead of stacked. **Same data, different stackId, different chart.** That's the pattern: choose the shape your data fits naturally, then let small props turn it into different visualizations.

Reshape #2: per-bar colors with `<Cell>`

The “Pick Distribution” chart for the Final has a special twist: each bar should be **green if the player picked is Wu Yize, gray otherwise**. Recharts lets you do that by mapping `<Cell>` children:

```

<Bar dataKey="count" radius={[6, 6, 0, 0]} ...>
  {finalPickData.map((d, i) => (
    <Cell key={i} fill={d.name === 'Wu Yize' ? '#16A34A' : '#9CA3AF'} />
  ))}
</Bar>

```

A `<Cell>` is a per-row override. The `<Bar>` defines the default behavior; the `<Cell>` for row `i` overrides the fill. There's one `<Cell>` per data row, in the same order. Without `<Cell>`, every bar is the same color. With `<Cell>`, every bar is whatever you want.

That conditional `fill={d.name === 'Wu Yize' ? '#16A34A' : '#9CA3AF'}` is the entire mechanism by which the chart highlights “Wu Yize was right.” Same pattern

works for any “highlight one thing” chart.

Reshape #3: pivot from “team picks” to “match agreement”

The most interesting chart on the page is `<LeagueAgreementChart>`. The question it answers is: “For each match in this round, of the 8 teams, how many picked the actual winner?” That’s a **pivot** — the source data is per-team-per-round, but the chart needs per-match-per-round.

```
const buildAgreementData = (matches, pickKey) => matches.map((m, i) => {
  // ...
  if (m.winner) {
    const correct = teams.filter(t => {
      const pick = pickKey === 'final' ? t.final : (t[pickKey] ? t[pickKey][i] : null);
      return pick === m.winner;
    }).length;
    return { match: matchLabel, correct, wrong: 8 - correct, pending: 0, finished: true };
  } else {
    // pre-tournament: split picks between p1 and p2
    const p1Backers = teams.filter(t => /* same dance */).length;
    const p2Backers = teams.filter(t => /* same dance */).length;
    return { match: matchLabel, p1Backers, p2Backers, pending: 8, finished: false };
  }
});
```

Two important things:

- **Same function returns two different shapes** depending on whether the match is finished. If finished: { match, correct, wrong, finished: true }. If pending: { match, p1Backers, p2Backers, finished: false }. The chart conditionally renders different `<Bar>` series based on `isPendingRound = round.data.every(d => !d.finished)`.
- **The pivot is index-based.** `t[pickKey][i]` reaches into team `t`’s pick array for round `pickKey` at index `i`. That works because of Lesson 1’s invariant — the `i`-th pick is for the `i`-th match in that round. Without that invariant this whole function would need a `find()` against `pick.matchId`, and it would have been twice as long.

Internal tab state inside a chart

`<LeagueAgreementChart>` is the only chart that has its own `useState`:

```
function LeagueAgreementChart({ rounds }) {  
  const [active, setActive] = useState('r1');  
  const round = rounds.find(r => r.id === active) || rounds[0];  
  // ...  
}
```

Why is its tab state local rather than lifted to `<SnookerFantasyLeague>`? Because **nothing outside this chart cares which round is active inside it**. It's a UI-internal toggle, not a piece of cross-cutting application state. The rule from Lesson 4 still holds: state lives at the lowest common ancestor of its readers and writers. Here both are inside `<LeagueAgreementChart>` itself, so the state is local. **Tab state isn't always app state**. Knowing the difference is half the React mental model.

Vibe prompt you would have used

"Add an analytics tab. Three charts at the top: (1) a stacked bar chart of points per team across R1/R2/QF/SF/Final using Recharts, X-axis is team name, stack each round in a different color. (2) A horizontal bar chart of pick accuracy %, one bar per round per team. (3) A bar chart of how many of 8 teams picked each Finalist; highlight the bar for the actual champion in green using `<Cell>`. Below those, a single 'League Agreement' chart with its own internal round-toggle (R1, R2, QF, SF, Final) – for each match in that round, show two stacked bars: green for teams who picked correctly, red for teams who picked wrong. Pull the team data into a fresh array shaped exactly how Recharts expects (`teams.map(...)` returning `{ team, R1, R2, ... }`); don't try to pass the team objects directly. Three insight cards beneath summarising leader, best round, champion."

CHECK YOURSELF

1. If you remove the `stackId="a"` prop from every `<Bar>` in the points-breakdown chart, what changes visually?
2. Why does `<LeagueAgreementChart>` keep its own `useState` instead of putting `active` in the top-level `<SnookerFantasyLeague>`?
3. The agreement-data builder has two return shapes (finished vs pending). What does

the chart code use to decide which `<Bar>` series to render — and what would happen if a single round had a mix of finished and unfinished matches?

Lesson 7 — The Evolution (what the git log would have shown)

Vibe-coded apps grow in layers, not waves. The current 1,654-line file didn't appear in one prompt. It accreted over (probably) a few dozen sessions across the two-week tournament. Here's what the git log would have looked like, reconstructed from the artifacts in the file. For each milestone, the **vibe prompt** that would have triggered the change.

Milestone 1 — “Just give me the standings”

Pre-tournament. R1 hasn't started. You have the bracket. You have eight friends' picks. You want a single screen that shows team totals.

“Give me one React component called `<StandingsTab>`. Render a table of 8 teams sorted by total points. Hard-code the `TEAMS` array and a `ROUND1_MATCHES` array (16 entries) above the component. Total points are 3 per correct R1 pick, 1 per wrong. Just R1 for now — the other rounds don't exist yet. White background, big readable numbers, no styling beyond a green header.”

The codebase at this point is **maybe 200 lines**. One file, one component, two constants, one `scorePick` function, one inline `.reduce()` for totals. There is no Matches tab, no Predictions tab, no `useState`, no `nav`. **This is the version you ship to your eight friends so they stop asking you “who's winning.”**

Milestone 2 — “I want to see the matches themselves”

Round 1 is half-finished. People want to know which matches are done and what the scores were.

“Add a second screen called Matches. It shows match cards, one per match, in a grid: player names, the score ('10-7' etc.), and a 'FINISHED' badge if the match has a winner. If there's no winner yet, show 'NOT FINISHED' with a dashed gray border. Make a top-of-page tab bar with two buttons: Standings and Matches. Use `useState` to switch between them.”

This is the prompt that introduces `useState` to the codebase. It also introduces the *concept of two screens that share data*, which is the single biggest jump in React complexity for a vibe coder. After this milestone the file is around **400 lines** and has its first orchestrator pattern.

Milestone 3 – “I want to compare predictions side by side”

End of Round 1. The standings tab tells you the *total*, but doesn’t show *whose pick was wrong*. You add a third screen.

“Add a third tab called Predictions. Two views: a comparison matrix showing every match as a row and every team as a column, with the team’s pick in each cell, green if correct and red if wrong. And a single-team card view, where clicking a team in the standings drops you here with that team selected. Round buttons inside the predictions tab to filter R1/R2/QF/SF.”

This is the prompt that introduces the *cross-tab interaction* pattern from Lesson 4 (clicking a team row in Standings switches to Predictions). It also introduces nested view state (`view: 'matrix' | 'team'` lives inside `<PredictionsTab>`, not at the top).

Milestone 4 – “Round 2 happened, the bracket shrunk”

Round 1 is done. Round 2 results trickle in. You add `ROUND2_MATCHES` and a `r2` array on every team. The whole `calculateTeamScores` function gets duplicated for R2:

“Round 2 is happening. Add `ROUND2_MATCHES` (8 entries) above `ROUND1_MATCHES`. Add `r2: [ 'Zhao Xintong', 'Shaun Murphy', ... ]` (8 picks) to every team. Update `calculateTeamScores` to also walk `r2` and return `{ r1, r2, total: r1 + r2, r1Details, r2Details }`. Add R2 columns to the standings table next to R1, and a Round 2 button to the Matches tab and the Predictions matrix.”

This is the prompt that creates the **duplication-by-round** pattern that runs through the whole file: every round needs a constant, a pick array, a totals field, a details field, a tab button, a matrix column. By the time you get to the Final, that pattern has fired five times. A more-architected codebase would parameterize it (`ROUNDS = [ 'r1', 'r2', 'qf', 'sf', 'final' ]` with a single loop). The vibe-coded version copies the pattern five times. Both are fine for an 8-team league.

Milestone 5 – “Show me a player’s tournament journey”

Mid-tournament, QF in flight. The Standings + Matches + Predictions trio is the league-level view. But you want to be able to ask, “How is Wu Yize doing? Which teams backed him? Did they get points?” That’s the Players tab.

“Add a Players tab. Grid of 32 cards, one per player from `PLAYER_INFO`. Filter buttons at the top: Still In, Eliminated, All. Click a card to open a deep player detail view: header with the player’s name + flag + status, a 5-step tournament path (`R1` → `R2` → `QF` → `SF` → `F`) with each step colored by won/lost/live/na, a stats row, and a per-match analysis showing which 8 teams backed this player vs the opponent in each round. The opponent’s name should be clickable so you can hop to their detail view too.”

This is the most ambitious single prompt in the project. It introduces **navigation by name** (selecting a player by string instead of by index), the **stillStanding set computed from QF + SF + Final results**, and the **PathStep + PathArrow** components from Lesson 5. After this milestone the file is ~1,200 lines.

Milestone 6 – “I want pretty charts”

Late tournament. With most matches in, you want trend visuals — points by round, accuracy by round, agreement on each match.

“Add an Analytics tab using Recharts. Three charts up top in a grid: stacked bars for total points (`R1+R2+QF+SF+Final`), horizontal bars for accuracy %, vertical bars for the Finalist pick distribution with the actual champion’s bar in green. Below them a ‘League Agreement’ chart with its own round-toggle showing per-match correct-vs-wrong stacks. Three colored insight cards at the bottom summarising leader, best R1, champion.”

This is where the file picks up Recharts as a dependency. The data-reshape pattern from Lesson 6 starts here.

Milestone 7 – “Wu Yize won. Crown him.”

Tournament ends. The Final result is in: Wu Yize 18, Murphy 17. The codebase needs to acknowledge that the champion is decided and the league is done.

“Wu Yize won the Final 18-17. Update `FINAL_MATCH` to set winner: ‘Wu Yize’. In

the hero header, replace the 'tournament in progress' badge with '🏆 WU YIZE – WORLD CHAMPION 2026 (18-17)'. In the standings, the Final Pick chip should be red for teams who picked Wu Yize and blue for teams who didn't. In the Players tab, the player card for Wu Yize gets a 🏆 WORLD CHAMPION badge; Murphy gets 🥈 RUNNER-UP. The other Crucible-eliminated players say OUT. Tournament-complete language everywhere it says 'tournament' so I can put it in past tense."

That last sentence is what lands the hard-coded 'Wu Yize' string we noted in Lesson 3. It's the trade-off: the prompt was easier to write because it referenced the actual champion by name; the cost was that the file is now tournament-specific. A purely time-symmetric version of this prompt would say *"compare against FINAL_MATCH[0].winner"* — identical UI, future-proof code. **A vibe prompt that uses a literal value is faster to write and produces faster code; the cost is portability.**

Putting the timeline together

If you read the file with this timeline in mind, you can almost see the seams between sessions. The five `*Details` arrays returned by `calculateTeamScores` each got added in their own session. The `<RoundButton>` and `<PicksList>` components were added at slightly different times — the round button has a status prop with literal English (`'FINISHED'`, `'🏆 WU WINS 18-17'`), which is unmistakably end-of-tournament wording, while `<PicksList>` is generic (it would render the same on day 1). **The newer code is more specific to "the tournament is over;" the older code is more generic to "any tournament."**

Vibe prompt you would have used (the meta-pattern)

There isn't a single prompt for the whole evolution — that's the point. There's one prompt per milestone, and **each prompt is shaped like an addition, not a redesign:**

"Add X to Y showing Z."

Read back over Milestones 1–7. Every one starts with "Add" or "Update" — never "Refactor everything" or "Make it nicer." That's the discipline that keeps the file growing instead of churning.

CHECK YOURSELF

1. The Final Pick chip's red/blue coloring is a Milestone 7 artifact. What earlier file evidence (in any other component) tells you the codebase wasn't always tournament-end-aware?
 2. Why is each round's logic copy-pasted (one block for r1, one for r2, ...) instead of looped over ['r1', 'r2', 'qf', 'sf', 'final']? What would looping cost in readability for a vibe coder reading this file?
 3. The Players tab is by far the biggest single milestone in the timeline. Why is that a smell *only* if you have to ship by Friday — and why is it fine for this project?
-

Lesson 8 — The Shape of Your Prompts (meta-lesson on vibe coding)

You've now seen eight lessons of "the prompt I would have used." The whole point of those was to give you a private library of prompt templates that **demonstrably produced this codebase**. Here's the pattern, distilled.

The seven shapes of a good vibe prompt

After two weeks of building this app, your prompt library should look like this:

1. **The "Add a feature" prompt.** *"Add a [component / column / tab] to [location] that [does Z when X]."* Single new thing, single insertion point, single behavior. Example: *"Add a Form column at the right of the standings table that compares a team's R1 percentage to their R2 percentage and shows Rising / Falling / Steady with an arrow icon."*
2. **The "Tweak the visual" prompt.** *"In [component], change [X] when [Y]."* Surgical visual edits. No structural changes. Example: *"In the Standings table, make the rank circle gold for 1st, silver for 2nd, bronze for 3rd; replace 1st place's number with a 🥇 emoji."*
3. **The "Extract a sub-component" prompt.** *"This <Foo> is getting long. Extract [X piece] into a <Bar> component with props { a, b }. Don't change any visuals — just move the JSX."* The "don't change any visuals" clause is mandatory. Without it the LLM redesigns. Example: see Lesson 5's prompt about <TeamChip>, <Path-Step>, <PathArrow>.

4. **The “Wire two screens together” prompt.** *“When [component A] does X, set [parent state Y] and switch to [component B].”* Names the cross-tab interaction explicitly. The parent component is the one that has to be edited, even though the *description* is about the children. Example: *“When a row in <StandingsTab> is clicked, set `selectedTeam` and switch the active tab to Predictions.”*
5. **The “Reshape data for a chart” prompt.** *“Build a chart of X using Recharts. The data is `[teams.map(t => ({ ... }))]`. Bars / lines / cells map to fields `[a, b, c]`; color the bar for [special row] differently using `<CeLL>`.”* You spell out the data shape inside the prompt; that becomes the chart spec. Example: see Lesson 6’s analytics prompt.
6. **The “Add a column to an existing array of objects” prompt.** *“Add a `[fieldName]` field to every entry in `[constant]` and update `[scorePick / calculateTeamScores / something]` to read it.”* Common when the data model grows. The literal phrasing names the constant + the field name + the consumer that needs updating, all in one sentence.
7. **The “End-of-feature polish” prompt.** *“In `[view]`, when `[end-state condition is true]`, change `[strings / colors / badges]` to past tense / champion / closed-out wording.”* This is the Milestone 7 prompt shape. It bakes the final state into the UI.

Three bad prompts and why they’re bad

You’re going to write some of these. Recognise them:

- **“Make the standings better.”** Too vague. The LLM has to invent what “better” means. You’ll get bigger fonts, drop shadows, animation, maybe a redesigned card layout, and likely something visually nicer that subtly broke the data flow. If you can’t name *what* about the standings should change and *why*, don’t prompt yet — go look at it for another minute and decide.
- **“Use Redux for state.”** Premature abstraction. This app has **two** pieces of state at the top level. Redux’s overhead (actions, reducers, dispatch wiring, dev tools setup) costs more than it buys until your state is contended by a dozen-plus components. If a prompt names a *technology* before naming a *problem*, it’s almost always going to wreck the file. Useful version: *“`teams` is being prop-drilled into 3 levels and I want to keep it accessible without passing it through `<MatchesTab>` which doesn’t use it. What’s the smallest change?”* (The honest answer: don’t pass it through `<Match-`

esTab>. You're already there.)

- **“Refactor the whole codebase to use Tailwind classes instead of inline styles.”** A redesign disguised as a refactor. The visual richness in this app depends on dynamically computed styles (gradients with team-specific accent colors, conditional backgrounds, dynamic shadows). Tailwind can do most of that, but in JIT-compiled arbitrary-value mode that's noisier than the inline style is, and the conversion has to be done file-by-file with visual diffing. Useful version: *“In <StatCard>, replace the inline style on the outer div with Tailwind classes. Keep the dynamic gradient (bg-CoLoR) as inline because it's a runtime prop. Don't touch any other component.”* One file, one prompt, one verifiable diff.

The prompt-tree of this app

If you imagine the file as a tree of prompts, the trunk is Milestone 1's “Give me the standings” prompt. Every subsequent prompt is a branch off an existing branch — never the trunk. **You never re-prompt the standings table from scratch.** You always say *“in the standings, change X.”* That discipline is what keeps a vibe-coded app from becoming a churning rewrite machine.

The shape, again:

One feature. One file (or a small set of named files). One sentence per change. Avoid technology names. Avoid superlatives. Mention specific data fields and component names. Refer to behavior, not “polish.”

Final word

The thing this course was reverse-engineering wasn't just a 1,654-line file. It was the **prompting strategy** that grew that file without rewrites. If you internalize Lesson 4's “state up, callbacks down,” Lesson 5's “extract when you copy-paste twice,” Lesson 6's “data shape, not chart library,” and Lesson 8's “Add X to Y showing Z,” you can build the next app — pickleball ladder, World Cup pool, whatever — in maybe a third of the prompts.

That's the course. Now open `course/SNOOKER_FANTASY_LEAGUE_COURSE.md` next to `_source/SnookerFantasyLeague.jsx` and `components/SnookerFantasyLeague.tsx`, and read the three side-by-side. The `.jsx` is what you built. The course is how you would have built it. The `.tsx` is where it lives now.

CHECK YOURSELF

1. Look back at any vibe prompt you've written this month. Which of the seven shapes does it match? If none, what's missing — a target component name? a behavior? a data field?
2. Why is *"Use Redux for state"* almost always the wrong prompt for an app of this size, and what is the **right** prompt to write when you feel that itch?
3. The seven prompt shapes in this lesson all share one structural feature. What is it? (Hint: count how many of them name a specific component, file, or constant by name.)